
VISÃO DO PROBLEMA

CONTEXTO

O projeto SGO-Motos está inserido no contexto do gerenciamento de oficinas mecânicas e manutenção automotiva, setor que vem sendo impactado pela transformação digital e pela necessidade de maior organização dos processos internos. Atualmente, muitas oficinas ainda realizam o controle de clientes, motos, estoque e ordens de serviço de forma manual ou utilizando ferramentas limitadas, o que pode gerar perda de informações, atrasos e dificuldades no acompanhamento dos serviços realizados.

Os sistemas computacionais vêm transformando a forma como oficinas mecânicas administram suas atividades, permitindo automatizar processos, centralizar informações e melhorar a eficiência no atendimento aos clientes. Além disso, soluções digitais auxiliam no controle de estoque, no histórico de manutenção dos veículos e na organização das ordens de serviço.

Nesse cenário, o SGO-Motos surge como uma solução web desenvolvida para facilitar o gerenciamento das atividades da oficina, oferecendo uma plataforma simples, organizada e acessível para controle de clientes, motos, produtos e serviços realizados.

CARACTERIZAÇÃO DO PROBLEMA/OPORTUNIDADE

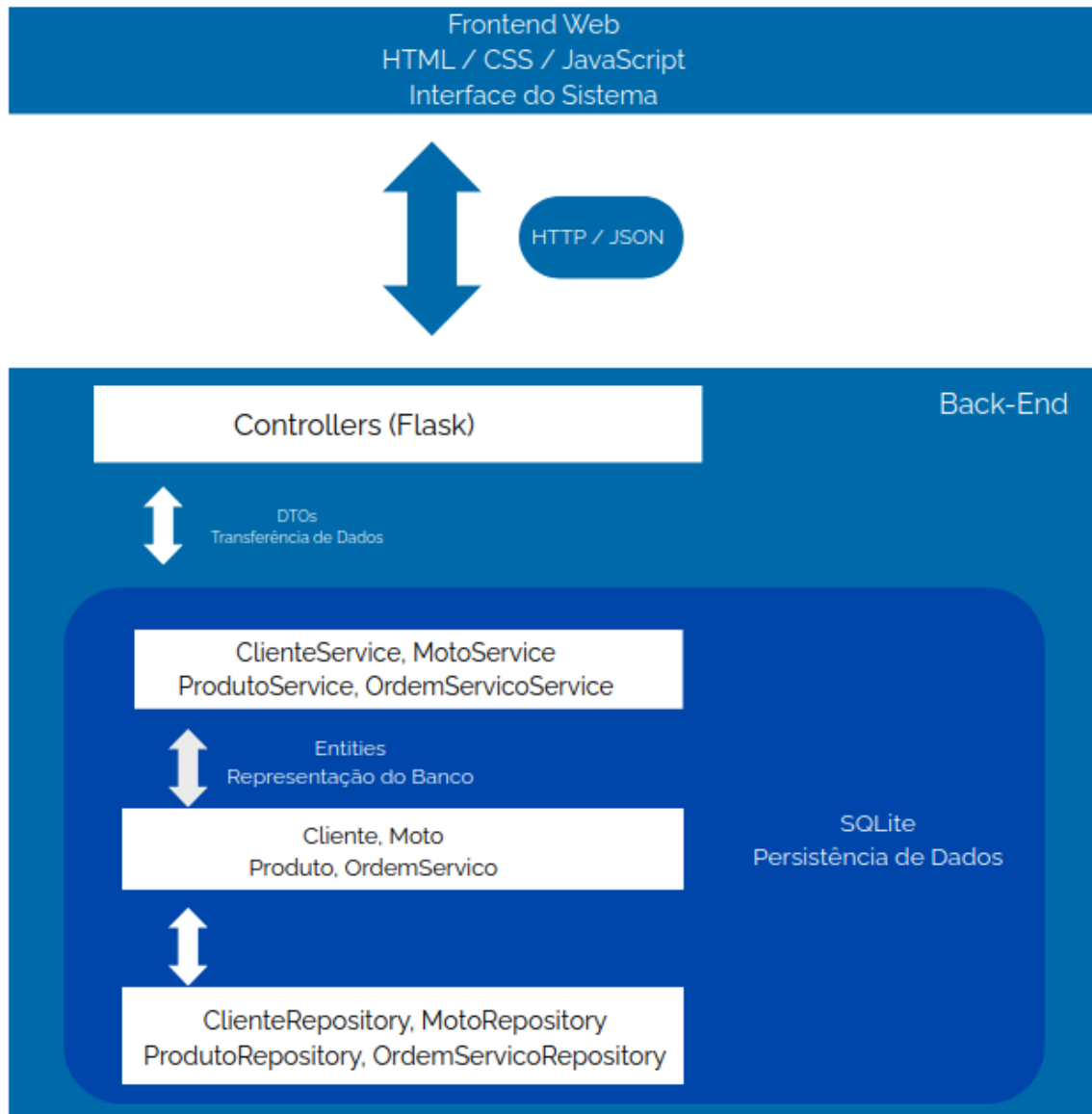
Atualmente, muitas oficinas mecânicas utilizam anotações manuais, planilhas ou sistemas limitados para controlar clientes, motos, produtos e ordens de serviço, o que pode gerar dificuldades na organização das informações, perda de dados e atrasos no gerenciamento dos serviços realizados.

Além disso, a ausência de um sistema centralizado dificulta o acompanhamento do histórico de manutenção dos veículos, o controle de estoque e o acesso rápido às informações da oficina, reduzindo a eficiência no atendimento aos clientes.

As soluções existentes nem sempre oferecem uma plataforma simples, acessível e organizada para pequenas e médias oficinas mecânicas. Nesse contexto, o SGO-Motos surge como uma oportunidade de centralizar o gerenciamento da oficina em um único sistema, automatizando processos e facilitando o controle das atividades realizadas diariamente.

ARQUITETURA

O sistema SGO-Motos será desenvolvido como uma aplicação web utilizando Python com Flask no backend, HTML, CSS e JavaScript no frontend, além de SQLite para persistência de dados. A aplicação será estruturada de forma modular, separando responsabilidades entre interface, lógica de negócio e armazenamento de dados.



- **Front-End**
 - A interface do sistema será desenvolvida utilizando HTML, CSS e JavaScript, sendo responsável pela interação com os usuários da oficina.

- As telas serão organizadas para facilitar o acesso às funcionalidades principais do sistema, como login, cadastro de clientes, motos, produtos e ordens de serviço.
- Os arquivos de interface serão organizados em pastas específicas, como:
 - templates: páginas HTML renderizadas pelo Flask.
 - static: arquivos estáticos, incluindo CSS, JavaScript e imagens do sistema.
- O frontend será responsável por enviar requisições ao backend e exibir as informações de forma simples e organizada.

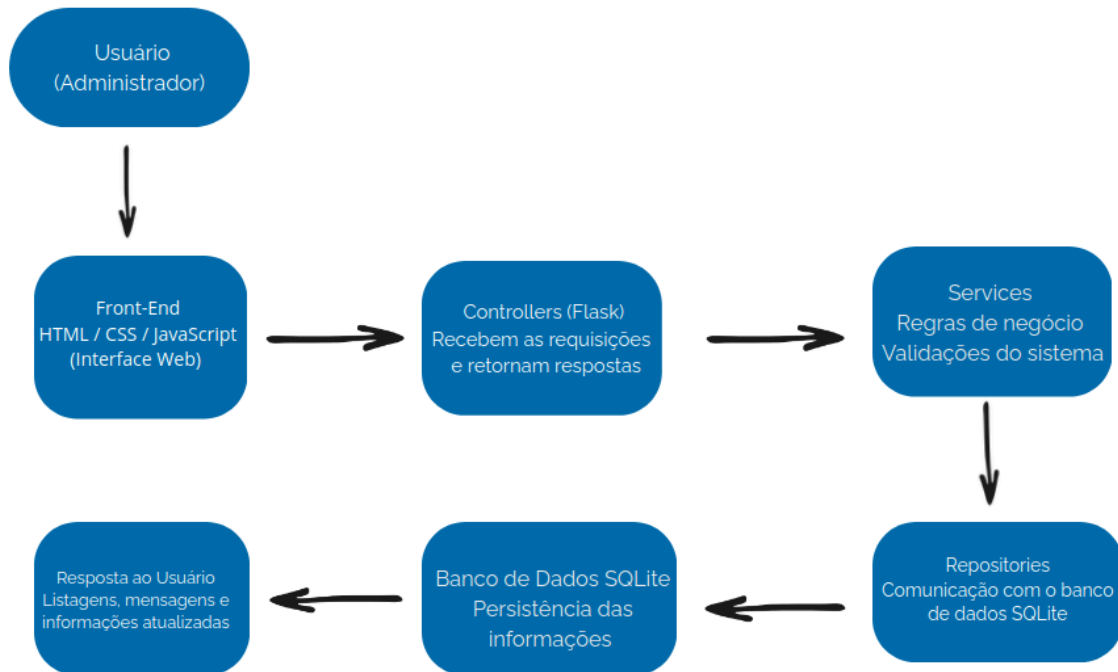
- **Back-End**

- O backend será organizado de forma modular, separando as funcionalidades do sistema em módulos relacionados às regras de negócio da aplicação, como clientes, motos, produtos e ordens de serviço.
- Cada módulo seguirá uma divisão em camadas para facilitar a manutenção, organização e escalabilidade do sistema:
 - Controller: responsável por receber as requisições HTTP e retornar as respostas ao usuário.
 - Service: responsável pela lógica de negócio e processamento das funcionalidades do sistema.
 - Repository: responsável pela comunicação com o banco de dados e manipulação das informações persistidas.
 - Entity: responsável por representar as entidades principais do sistema, como clientes, motos, produtos e ordens de serviço.
 - DTOs: responsáveis pela transferência de dados entre as camadas da aplicação, separando os dados utilizados nas requisições e respostas das entidades do sistema.
 - Exceptions: responsável por armazenar exceções personalizadas e tratamento de erros da aplicação.
- Além disso, o sistema possuirá uma estrutura compartilhada para armazenar classes utilitárias, funções reutilizáveis e configurações globais utilizadas em diferentes módulos da aplicação.

FLUXO DE DADOS

O fluxo de dados do sistema SGO-Motos ocorre entre a interface web desenvolvida em HTML/CSS e o backend em Python utilizando Flask, responsável pelo processamento das informações e comunicação com o banco de dados SQLite.

- **Requisição inicial:** o usuário acessa o sistema através do navegador web e realiza ações como login, cadastro de clientes, motos, produtos ou ordens de serviço.
- **Processamento da requisição:** as informações enviadas pela interface são recebidas pelas rotas do Flask, responsáveis por direcionar as requisições para as funcionalidades correspondentes do sistema.
- **Aplicação das regras de negócio:** após o recebimento da requisição, a camada responsável pela lógica do sistema realiza validações, processamento dos dados e aplicação das regras de negócio necessárias para cada funcionalidade.
- **Acesso ao banco de dados:** o sistema utiliza o SQLite para armazenar, buscar, atualizar e remover informações relacionadas aos usuários, clientes, motos, produtos e ordens de serviço.
- **Retorno da resposta:** após o processamento, o backend retorna as informações para a interface web, exibindo mensagens de confirmação, listagens ou resultados das operações realizadas pelo usuário.
- **Persistência das informações:** todos os dados manipulados no sistema permanecem armazenados no banco de dados, permitindo consultas e gerenciamento contínuo das atividades da oficina.
- **Tratamento no front-end:** a interface web recebe as respostas enviadas pelo backend e atualiza as informações exibidas ao usuário conforme a operação realizada. O sistema pode apresentar mensagens de confirmação, erros de validação, listagens atualizadas e informações relacionadas aos cadastros, estoque e ordens de serviço.



MODELO DE DOMÍNIO E ESTRUTURAS DE DADOS

O sistema SGO-Motos possui diferentes entidades e estruturas responsáveis pelo gerenciamento das informações da oficina mecânica, seguindo uma arquitetura organizada em camadas para facilitar a manutenção e evolução da aplicação.

- **Entidades:** representadas por classes responsáveis por modelar os principais elementos do sistema, como usuários, clientes, motos, produtos e ordens de serviço. Essas entidades correspondem diretamente às tabelas do banco de dados SQLite.
- **Serviços e regras de negócio:** a camada de services é responsável por aplicar as regras de negócio do sistema, validar informações e realizar o gerenciamento das operações relacionadas aos clientes, motos, estoque e ordens de serviço.
- **Repositórios:** responsáveis pela comunicação com o banco de dados, realizando operações de cadastro, consulta, atualização e remoção das informações persistidas.
- **Controllers:** responsáveis por receber as requisições HTTP enviadas pela interface web e retornar respostas estruturadas ao usuário, conectando a interface às funcionalidades do sistema.
- **DTOs:** utilizados para realizar a transferência de dados entre as camadas da aplicação, organizando as informações enviadas e recebidas pelo sistema.

- Estrutura relacional: o banco de dados é estruturado de forma relacional, permitindo o vínculo entre clientes, motos, produtos e ordens de serviço, garantindo integridade e organização das informações armazenadas.

PERSISTÊNCIA

O sistema SGO-Motos utiliza uma arquitetura de persistência centralizada, empregando o banco de dados SQLite para armazenamento das informações da aplicação.

- Banco de dados principal: o sistema utiliza SQLite como solução de persistência, sendo responsável por armazenar os dados relacionados aos usuários, clientes, motos, produtos, estoque e ordens de serviço.
- Organização dos dados: as informações são estruturadas em tabelas relacionais, permitindo manter vínculos entre entidades do sistema, como clientes e motos cadastradas, além das ordens de serviço associadas.
- Persistência das operações: todas as operações realizadas no sistema, como cadastro, atualização, consulta e remoção de informações, são persistidas no banco de dados para garantir integridade e disponibilidade dos dados.
- Integração com o backend: o backend desenvolvido em Flask realiza a comunicação com o banco SQLite através da camada de acesso a dados, responsável por executar operações de leitura e escrita no banco.
- Estrutura relacional: o modelo relacional adotado facilita a organização e gerenciamento das informações da oficina, permitindo consultas eficientes e maior controle sobre os dados armazenados.

SEGURANÇA E PRIVACIDADE

O sistema SGO-Motos adotará práticas de segurança para proteger os dados armazenados e controlar o acesso às funcionalidades da aplicação.

- Autenticação: o acesso ao sistema será realizado através de login com usuário e senha, permitindo identificar e validar os administradores cadastrados.
- Controle de acesso: determinadas funcionalidades do sistema estarão disponíveis apenas para usuários autenticados, garantindo maior segurança no gerenciamento das informações da oficina.
- Proteção de dados: os dados cadastrados no sistema serão armazenados de forma segura no banco de dados SQLite, evitando acesso não autorizado às informações.

-
- Validação de informações: o sistema realizará validações de entrada de dados para reduzir erros e evitar inconsistências durante o cadastro e gerenciamento das informações.

DESEMPENHO E CONFIABILIDADE

O sistema SGO-Motos deve garantir desempenho estável e eficiente, proporcionando uma boa experiência de uso durante o gerenciamento das atividades da oficina.

- Tempo de inicialização: a aplicação web deverá iniciar em poucos segundos após a execução do servidor Flask.
- Tempo de resposta: as operações realizadas pelo sistema devem apresentar respostas rápidas durante cadastros, consultas e atualizações de informações.
- Consumo de recursos: o sistema será desenvolvido de forma leve, permitindo sua execução em computadores com configurações básicas utilizadas em oficinas mecânicas.
- Confiabilidade: as informações cadastradas devem permanecer armazenadas corretamente no banco de dados, garantindo integridade e disponibilidade dos dados da oficina.

QUALIDADE E TESTES

O projeto SGO-Motos utiliza práticas de desenvolvimento para garantir organização, qualidade do código e estabilidade da aplicação.

- Controle de versão: utilização do Git e GitHub para versionamento do código-fonte e gerenciamento das alterações realizadas no projeto.
- Organização do código: separação da aplicação em camadas e módulos para facilitar manutenção, leitura e evolução do sistema.
- Testes manuais: realização de testes durante o desenvolvimento para validar funcionalidades como login, cadastro, gerenciamento de estoque e ordens de serviço.
- Validação de funcionalidades: verificação do funcionamento correto das operações de cadastro, atualização, consulta e remoção de informações no sistema.

DEPLOY

O deploy do sistema SGO-Motos será realizado em ambiente web, permitindo o acesso da aplicação através do navegador.

-
- **Back-end:** o sistema backend desenvolvido em Flask poderá ser hospedado em servidores web compatíveis com aplicações Python.
 - **Front-end:** a interface web será disponibilizada através do navegador, permitindo acesso às funcionalidades do sistema sem necessidade de instalação local por parte do usuário final.
 - **Banco de dados:** o SQLite será utilizado inicialmente como solução de persistência local para armazenamento das informações da aplicação.
 - **Evolução futura:** futuramente, o sistema poderá ser implantado em serviços de hospedagem em nuvem para ampliar disponibilidade e acessibilidade da aplicação.

DECISÕES ARQUITETURAIS

- **Separação em camadas:** o sistema SGO-Motos foi estruturado em camadas, separando interface, controle de requisições, regras de negócio e acesso ao banco de dados. Essa organização facilita a manutenção, reutilização de código e evolução da aplicação.
- **Uso de Controllers e Services:** os controllers são responsáveis apenas por receber e responder requisições HTTP, enquanto a lógica de negócio permanece centralizada na camada de services. Essa abordagem reduz acoplamento e melhora a organização do sistema.
- **Utilização de DTOs:** o sistema utiliza DTOs (Data Transfer Objects) para transferência de dados entre as camadas da aplicação, evitando acesso direto às entidades do banco de dados e tornando as respostas mais organizadas e seguras.
- **Organização modular:** as funcionalidades do sistema foram separadas em módulos relacionados às principais áreas da oficina, como clientes, motos, produtos e ordens de serviço. Essa divisão facilita futuras expansões e manutenção do projeto.
- **Componentização da interface:** no front-end, a estrutura foi organizada para reutilização de componentes visuais e páginas HTML, reduzindo duplicação de código e facilitando alterações futuras na interface do sistema.
- **Persistência simplificada:** foi escolhido o SQLite como banco de dados devido à simplicidade de integração, leveza e facilidade de utilização durante o desenvolvimento inicial do sistema.
- **Estratégia de testes:** serão realizados testes manuais durante o desenvolvimento para validar o funcionamento das funcionalidades principais do sistema, como autenticação, cadastro de clientes, controle de estoque e ordens de serviço.

-
- Controle de versão: o uso do GitHub permite acompanhar alterações realizadas no projeto, organizar versões da aplicação e facilitar o desenvolvimento colaborativo entre os integrantes da equipe.